



بسمه تعالی

آزمایشگاه معماری کامپیوتر

دکتر سربازی آزاد

نرگس کاری

سروش داوران

گزارش شماره ۹

تاریخ: ۱۰ تیر ۱۴۰۵

شماره دانشجویی: ۴۰۲۱۱۰۸۲۱

شماره دانشجویی: ۴۰۲۱۰۵۹۸۷

طراحی واحد کنترل ریزبرنامه‌سازی شده

هدف آزمایش

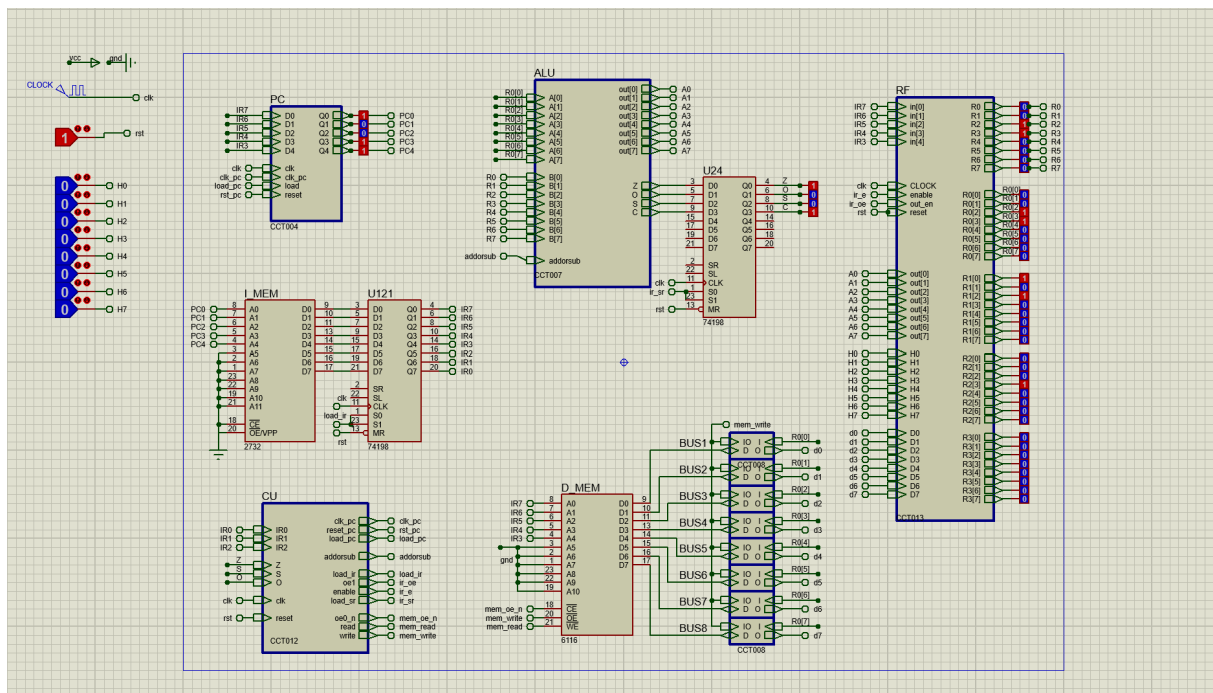
هدف این آزمایش، طراحی و پیاده‌سازی یک واحد کنترل ریزبرنامه‌سازی شده (Microprogrammed Control Unit) برای پردازنده طراحی شده در آزمایش‌های قبل است.

در آزمایش‌های ششم تا هشتم، سیگنال‌های کنترلی توسط یک واحد کنترل سخت‌سیمی تولید می‌شدند. در این آزمایش این واحد با یک کنترل‌کننده ریزبرنامه‌ای جایگزین شد تا فرآیند تولید سیگنال‌های کنترلی از طریق اجرای ریزدستورهای ذخیره‌شده در حافظه ریزبرنامه انجام شود.

پس از طراحی واحد کنترل، اجرای برنامه جمع جملات دنباله فیبوناچی روی پردازنده بررسی شده و صحت عملکرد سیستم مورد ارزیابی قرار گرفت.

معماری کلی سیستم

شکل ۱ معماری کامل پردازنده طراحی شده در محیط Proteus را نشان می‌دهد. این پردازنده از واحدهایی شامل شمارنده برنامه، حافظه دستور، ثبات دستور، فایل ثبات‌ها، واحد محاسبات و منطق، حافظه داده و واحد کنترل ریزبرنامه‌ای تشکیل شده است.



شکل ۱: معماری کلی پردازنده

همان‌طور که در شکل ۱ مشاهده می‌شود، تمامی بلوک‌های پردازنده نسبت به آزمایش هشتم بدون تغییر باقی مانده‌اند و تنها واحد کنترل سخت‌سیمی با یک واحد کنترل ریزبرنامه‌ای جایگزین شده است. به همین دلیل در این گزارش تنها جزئیات مربوط به طراحی واحد کنترل جدید مورد بررسی قرار می‌گیرد.

روند طراحی

از آنجا که معماری کلی پردازنده در آزمایش‌های ششم تا هشتم طراحی و پیاده‌سازی شده بود، در این آزمایش نیازی به تغییر سایر بخش‌های پردازنده از جمله فایل ثبات‌ها، واحد محاسبات و منطق، حافظه دستور و حافظه داده وجود نداشت. بنابراین تمرکز این آزمایش بر جایگزینی واحد کنترل سخت‌سیمی با یک واحد کنترل ریزبرنامه‌ای بود.

در ابتدا ریزدستورهای مورد نیاز برای مراحل واکشی، رمزگشایی و اجرای هر یک از دستورهای ماشین استخراج شدند. سپس این ریزدستورها به گونه‌ای طراحی شدند که علاوه بر تولید سیگنال‌های کنترلی مورد نیاز پردازنده، امکان اجرای پرش‌های شرطی و بدون شرط نیز فراهم شود.

پس از تعیین قالب ریزدستورها، محتوای حافظه ریزبرنامه به صورت فایل‌های دودویی تولید شد. این فایل‌ها در دو حافظه EPROM 2732 بارگذاری شدند تا در کنار یکدیگر یک حافظه ریزبرنامه ۱۶ بیتی را تشکیل دهند. در نهایت واحد کنترل طراحی شده در مدار اصلی پردازنده قرار گرفت و صحت عملکرد آن با اجرای برنامه جمع جملات دنباله فیبوناچی در محیط Proteus مورد بررسی قرار گرفت.

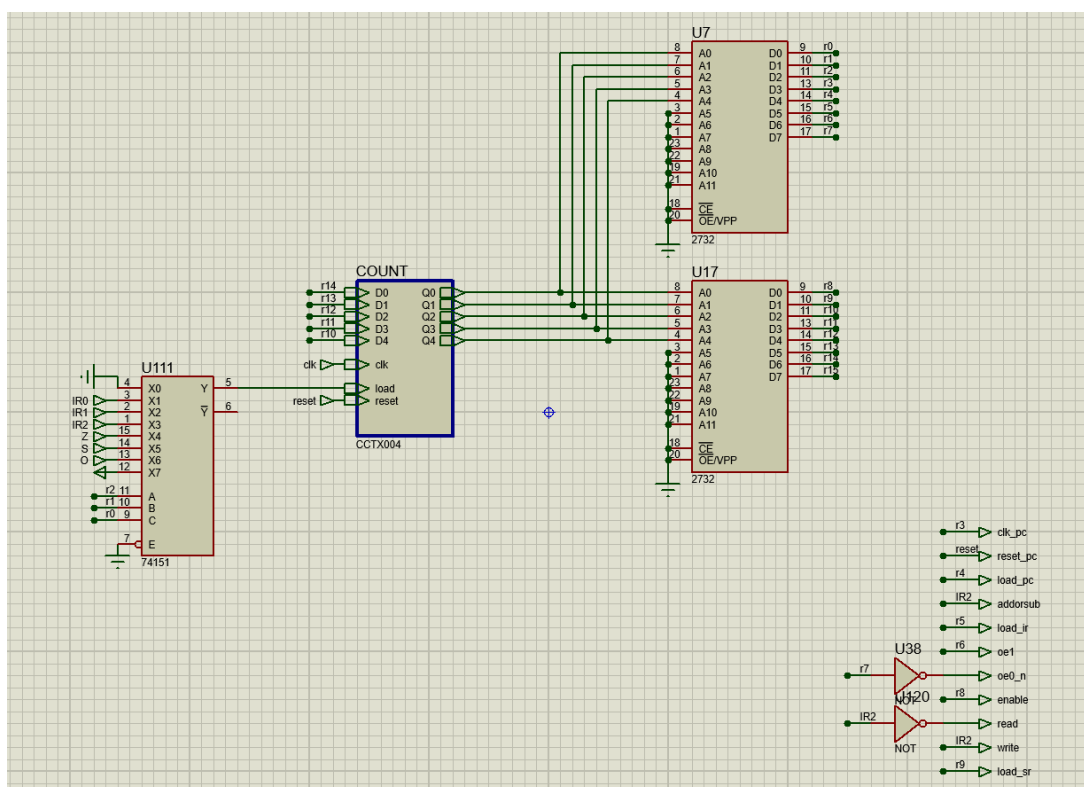
طراحی واحد کنترل ریزبرنامه‌ای

ساختار کلی

به منظور جایگزینی واحد کنترل سخت‌سیمی آزمایش‌های قبل، یک واحد کنترل ریزبرنامه‌ای طراحی شد. در این روش، کلیه سیگنال‌های کنترلی مورد نیاز پردازنده به جای تولید توسط مدارهای ترکیبی، در قالب ریزدستورها داخل یک حافظه فقط‌خواندنی ذخیره می‌شوند. واحد کنترل طراحی شده از چهار بخش اصلی تشکیل شده است:

- شمارنده ریزبرنامه (Micro Program Counter)
- حافظه ریزبرنامه
- منطق انتخاب شرط پرش
- منطق تولید سیگنال‌های کنترلی

شکل ۲ ساختار کامل واحد کنترل را نشان می‌دهد.



شکل ۲: ساختار واحد کنترل ریزبرنامه‌ای

شمارنده ریزبرنامه

برای آدرس‌دهی ریزحافظه از یک شمارنده پنج‌بیتی استفاده شد که نقش Micro Program Counter (μPC) را بر عهده دارد.

در هر سیکل ساعت، این شمارنده آدرس ریزدستور بعدی را تولید می‌کند. در حالت عادی شمارنده به صورت ترتیبی افزایش یافته و ریزدستور بعدی اجرا می‌شود. در صورت فعال شدن سیگنال Load، مقدار جدیدی از طریق منطق پرش در شمارنده بارگذاری شده و اجرای ریزبرنامه از آدرس جدید ادامه پیدا می‌کند.

حافظه ریزبرنامه

هر ریزدستور ذخیره‌شده در حافظه ریزبرنامه شامل ۱۶ بیت است. از این ۱۶ بیت، ۱۰ بیت مستقیماً سیگنال‌های کنترلی پردازنده را تولید می‌کنند، ۵ بیت برای تعیین آدرس ریزدستور بعدی استفاده می‌شوند و یک بیت باقی‌مانده نوع عملیات پرش را مشخص می‌کند.

به دلیل محدودیت اندازه خروجی هر ROM، حافظه ریزبرنامه توسط دو حافظه EPROM نوع ۲۷۳۲ پیاده‌سازی شد. هر EPROM هشت بیت از هر ریزدستور را تولید می‌کند و خروجی دو حافظه در کنار یکدیگر یک ریزدستور ۱۶ بیتی را تشکیل می‌دهند.

اگرچه هر حافظه ظرفیت ۴۰۹۶ خانه را در اختیار قرار می‌دهد، در این آزمایش تنها ۱۶ خانه ابتدایی حافظه برای ذخیره ریزدستورها مورد استفاده قرار گرفته است.

تولید سیگنال‌های کنترلی

خروجی حافظه ریزبرنامه مستقیماً بیشتر سیگنال‌های کنترلی پردازنده را تولید می‌کند. با این حال برخی از سیگنال‌ها در مدار اصلی به صورت Active Low تعریف شده‌اند. به همین دلیل برای تولید این سیگنال‌ها از گیت‌های NOT استفاده شد تا سطح منطقی مناسب برای فعال‌سازی این بخش‌ها فراهم شود. در طراحی حاضر، سیگنال‌های مربوط به فعال‌سازی خروجی ثبات‌ها و همچنین کنترل حافظه با استفاده از این روش تولید شده‌اند.

منطق پرش ریزبرنامه

برای انتخاب مسیر اجرای ریزبرنامه از یک مالتی‌پلکسر 74151 استفاده شده است. سه بیت Micro OpCode که از ریزدستور استخراج می‌شوند، ورودی انتخاب این مالتی‌پلکسر را تشکیل می‌دهند. بر اساس مقدار این سه بیت، یکی از سیگنال‌های IR0، IR1، IR2 و یا پرچم‌های وضعیت Z، S و O انتخاب می‌شود.

در صورتی که شرط انتخاب‌شده برقرار باشد، خروجی مالتی‌پلکسر فعال شده و سیگنال Load برای Micro Program Counter تولید می‌شود. در نتیجه، به جای افزایش ترتیبی شمارنده، آدرس پرش موجود در ریزدستور در μ PC بارگذاری شده و اجرای ریزبرنامه از آدرس جدید ادامه پیدا می‌کند.

در حالتی که نوع ریزدستور برابر Normal باشد، سیگنال Load غیرفعال باقی‌مانده و μ PC به صورت ترتیبی ریزدستور بعدی را اجرا می‌کند. به همین ترتیب، پرش‌های شرطی بر اساس بیت‌های دستور ماشین (IR0، IR1، IR2) و همچنین پرچم‌های وضعیت (Z، S، O) پیاده‌سازی شده‌اند و در حالت Jump Always بارگذاری آدرس جدید بدون توجه به هیچ شرطی انجام می‌شود.

ریزدستورهای طراحی شده

پس از طراحی ساختار واحد کنترل، تمامی مراحل اجرای پردازنده به صورت ریزدستورهایی در حافظه ریزبرنامه ذخیره شدند. هر ریزدستور مجموعه‌ای از سیگنال‌های کنترلی لازم برای انجام یک ریزعملیات را تولید می‌کند و علاوه بر آن، نحوه انتقال به ریزدستور بعدی را نیز مشخص می‌نماید.

در طراحی انجام‌شده، برای هر ریزدستور یک قالب ۱۶ بیتی در نظر گرفته شد که شامل بیت‌های کنترل، نوع پرش و آدرس مقصد پرش است. با اجرای متوالی این ریزدستورها، مراحل واکشی، رمزگشایی و اجرای دستورات ماشین انجام می‌شود.

جدول ۱: قالب ریزدستور ذخیره‌شده در حافظه ریزبرنامه

بخش ریزدستور	وظیفه
Micro OpCode	تعیین نوع پرش
بیت‌های کنترل	تولید سیگنال‌های کنترلی پردازنده
آدرس پرش	تعیین مقصد پرش در صورت برقرار بودن شرط

جدول ۲: ریزدستورهای طراحی شده

آدرس	نام مرحله	عملکرد
۰	Begin	واکشی دستور از حافظه
۱	Decode	تشخیص نوع دستور ماشین
۳	Add/Sub	اجرای عملیات حسابی و منطقی
۴	Load/Store	تعیین نوع دسترسی حافظه
۵	Load	خواندن داده از حافظه
۶	Store	نوشتن داده در حافظه
۷ تا ۱۶	Branch	اجرای انواع پرش‌های شرطی و بدون شرط

روند اجرای ریزبرنامه

پس از روشن شدن سیستم، اجرای ریزبرنامه از آدرس صفر آغاز می‌شود. در این مرحله دستور جاری از حافظه برنامه واکشی شده و در ثبات دستور (Instruction Register) قرار می‌گیرد.

پس از پایان مرحله واکشی، کنترل به مرحله Decode منتقل می‌شود. در این مرحله با بررسی بیت‌های دستور ماشین، نوع عملیات مشخص شده و مسیر مناسب اجرای ریزبرنامه انتخاب می‌شود.

در صورت تشخیص دستور جمع یا تفریق، کنترل به ریزدستور Add/Sub منتقل شده و سیگنال‌های لازم برای فعال‌سازی واحد ALU تولید می‌شوند.

در صورتی که دستور از نوع بارگذاری یا ذخیره‌سازی باشد، ابتدا مرحله Load/Store اجرا شده و سپس بر اساس نوع عملیات، یکی از مراحل Load یا Store فعال می‌شود.

برای دستورهای پرش نیز ریزبرنامه وارد بخش Branch می‌شود. در این قسمت، با استفاده از مالتی‌پلکسر 74151 یکی از شرط‌های وابسته به بیت‌های دستور یا پرچم‌های وضعیت انتخاب شده و در صورت برقرار بودن شرط، آدرس

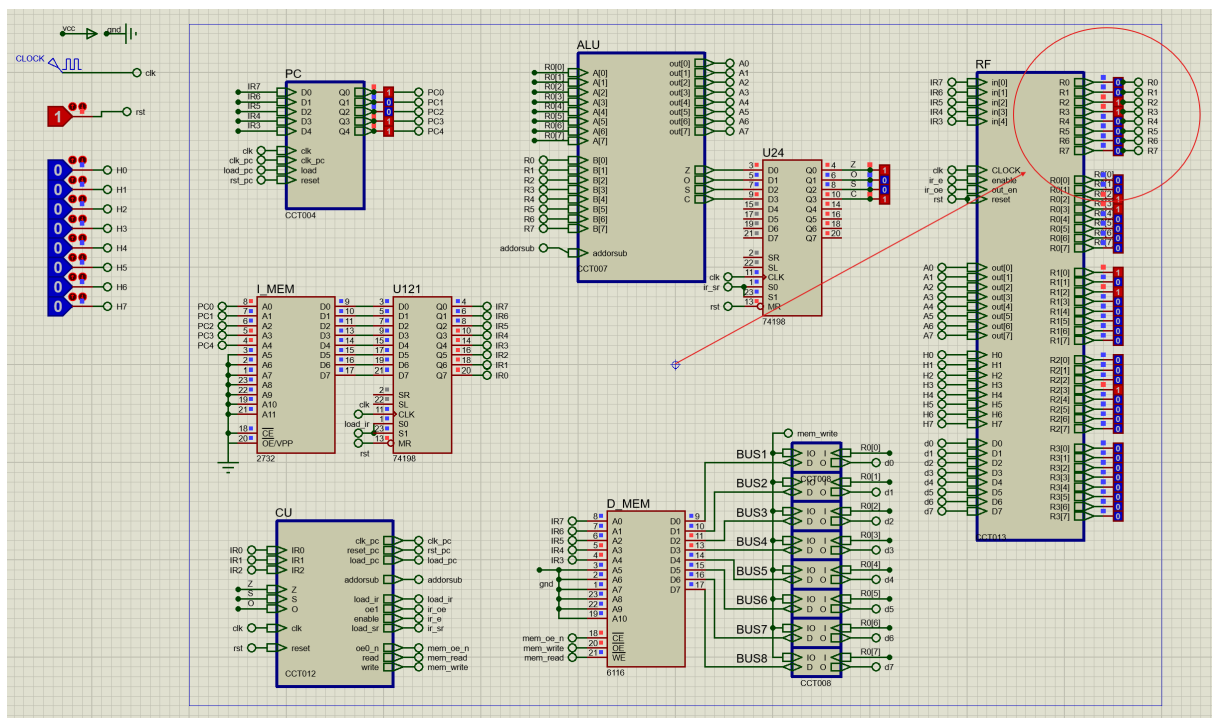
جدید در Micro Program Counter بارگذاری می‌شود؛ در غیر این صورت اجرای ترتیبی ریزدستورها ادامه خواهد یافت.

آزمون عملکرد و نتایج

به منظور ارزیابی عملکرد واحد کنترل ریزبرنامه‌ای، سه برنامه مختلف روی پردازنده اجرا شد. این برنامه‌ها با هدف بررسی عملکرد بخش‌های مختلف پردازنده شامل واحد محاسبات و منطق، دسترسی به حافظه و دستورهای پرش طراحی شده‌اند. نتایج حاصل از اجرای هر برنامه در ادامه ارائه شده است.

اجرای برنامه جمع پنج جمله اول دنباله فیبوناچی

در نخستین آزمون، برنامه‌ای جهت محاسبه مجموع پنج جمله اول دنباله فیبوناچی در حافظه برنامه قرار گرفت. این برنامه با استفاده از دستورات محاسباتی، دسترسی به ثبات‌ها و پرش‌های شرطی، مجموع جملات را محاسبه کرده و نتیجه را در پایان اجرا تولید می‌کند. پس از اجرای کامل برنامه، مقدار نهایی برابر با ۱۲ به دست آمد که مطابق مقدار مورد انتظار است. این نتیجه نشان می‌دهد که واحد کنترل ریزبرنامه‌ای تمامی مراحل واکشی، رمزگشایی، اجرای عملیات حسابی و کنترل جریان برنامه را به درستی انجام داده است.

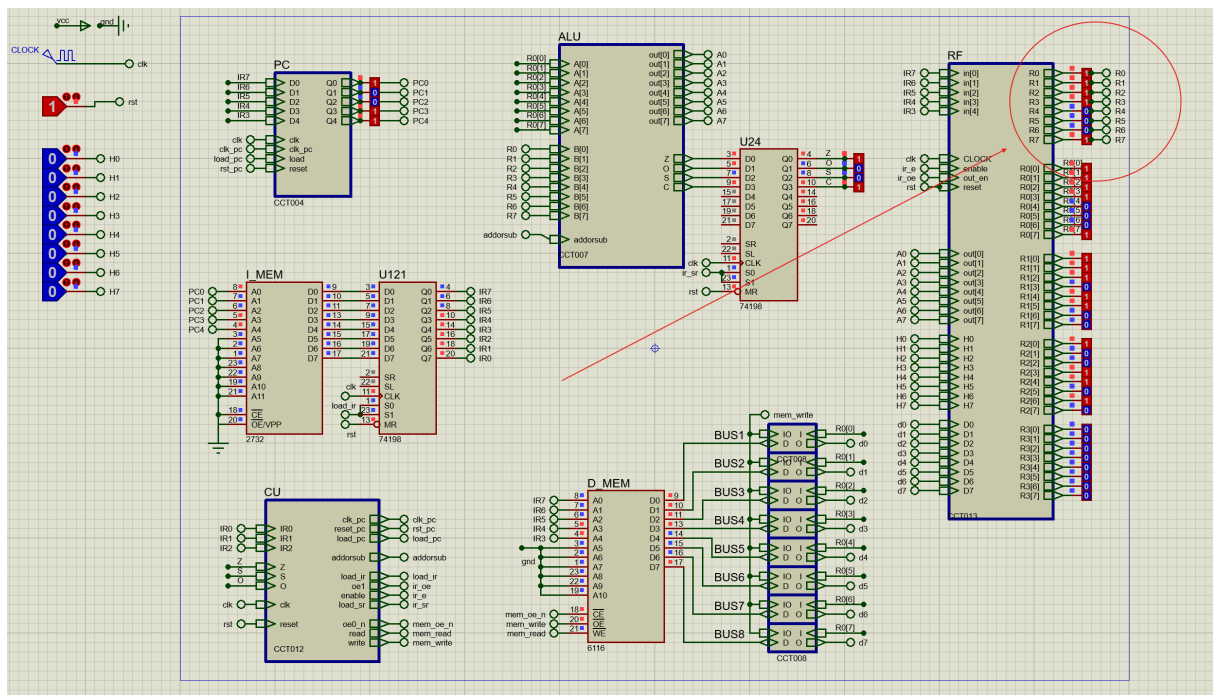


شکل ۳: اجرای برنامه جمع پنج جمله اول دنباله فیبوناچی

اجرای برنامه جمع ده جمله اول دنباله فیبوناچی

به منظور ارزیابی عملکرد واحد کنترل ریزبرنامه‌ای در اجرای برنامه‌های شامل حلقه و پرش‌های شرطی، برنامه محاسبه مجموع ده جمله اول دنباله فیبوناچی در حافظه برنامه بارگذاری و اجرا شد. این برنامه علاوه بر استفاده از عملیات جمع، در هر تکرار از دستورهای پرش شرطی برای کنترل حلقه نیز بهره می‌برد؛ بنابراین اجرای صحیح آن نشان‌دهنده عملکرد صحیح بخش‌های مختلف واحد کنترل است.

همان‌طور که در شکل ۴ مشاهده می‌شود، برنامه بدون بروز خطا تا پایان اجرا شده و نتیجه مورد انتظار تولید گردید. این موضوع نشان می‌دهد که ریزدستورهای مربوط به مراحل واکنشی، رمزگشایی، عملیات حسابی و کنترل جریان برنامه به درستی طراحی و در حافظه ریزبرنامه ذخیره شده‌اند.



شکل ۴: اجرای برنامه جمع ده جمله اول دنباله فیبوناچی

اجرای برنامه جمع دو عدد ۶۴ بیتی

در آزمون سوم، برنامه جمع دو عدد ۶۴ بیتی مطابق صورت آزمایش هشتم اجرا شد. در این برنامه، یکی از اعداد به صورت مقدار ثابت با تمامی بیت‌های برابر یک در حافظه داده قرار داشت و عدد دوم نیز پیش از اجرا در خانه‌های حافظه مقداردهی شد.

به دلیل محدودیت پهنای داده پردازنده، عملیات جمع به صورت چندمرحله‌ای و روی بخش‌های ۸ بیتی انجام شده و انتقال نقلی بین مراحل متوالی مدیریت شد. اجرای صحیح این برنامه نشان داد که علاوه بر عملیات محاسباتی، دسترسی به حافظه و مدیریت نقلی بین بایت‌های متوالی نیز توسط واحد کنترل ریزبرنامه‌ای به درستی انجام می‌شود.

جمع‌بندی نتایج

اجرای موفق هر سه برنامه نشان داد که واحد کنترل ریزبرنامه‌ای طراحی شده قادر است تمامی مراحل اجرای دستورها شامل واکشی، رمزگشایی، عملیات محاسباتی، دسترسی به حافظه و پرس‌های شرطی را بدون وابستگی به منطق کنترل سخت‌سیمی مدیریت کند. همچنین تطابق خروجی برنامه‌ها با نتایج مورد انتظار، صحت عملکرد ریزدستورهای ذخیره‌شده در حافظه ریزبرنامه و منطق کنترل طراحی شده را تأیید می‌کند.